Day 22 — Splitter

1. What Is a Splitter?

> **Splitter:** split one message into **many small messages** — that is the use of the Splitter. If you are getting **multiple records in one message, or a huge payload**, and you want to split it into multiple messages, you use a Splitter.

Example Payload from Class

```
<root>
  <employeeInfo>
    <employee>
      <EmpID>1</EmpID>
      <status>Active</status>
    </employee>
    <employee>
      <EmpID>2</EmpID>
      <status>InActive</status>
    </employee>
  </employeeInfo>
</root>
```

> **XML is case-sensitive:** in your sketch the opening tag is `<EmpID>` but the closing tag is written `</Empid>` (and one closing tag reads `</employees>` instead of `</employee>`). Opening and closing tags must match **exactly**, including capital letters — otherwise the XML is invalid and the Splitter (or any XML step) will fail. I've corrected it above.

2. Why Do We Need It?

- **Recap from Day 21:** if we send the above XML payload, the **Router cannot handle it** (it decides once per message, not per record). **To overcome that situation we use the Splitter palette function.**
- **Split the data:** you're getting 2 employees' information (or 100) combined in one payload → **split that one message into 2 messages** (or 100).

> **Requirement ①:** the target system accepts **only one record at a time** and cannot accept multiple records together — we have only one option: use the Splitter.
>
> **Real-time example ②:** sometimes we need to **update a record**. At a time **only one record** will update — so we need to send **1 record for each update**, and we **must use the Splitter.**

1 message (2 employees)	→	**Splitter**	→	Message 1 (Emp 1) Message 2 (Emp 2)	→	Router / Receiver

3. The Seven Splitter Types (Palette → Routing)

#	Splitter	Purpose (added)
①	**General Splitter**	Splits XML (by XPath) or text (by line) — keeps the enveloping elements
②	**Iterating Splitter**	Same, but **without** the enveloping elements
③	**EDI Splitter**	Splits an EDI interchange into individual EDI messages
④	**IDoc Splitter**	Splits a message containing several IDocs into single IDocs
⑤	**PKCS#7/CMS Splitter**	Splits signed/enveloped (PKCS#7/CMS) messages into their parts
⑥	**Tar Splitter**	Splits a TAR archive into its files
⑦	**Zip Splitter**	Splits a ZIP archive into its files

Class focused on the **General** and **Iterating** splitters — the two you'll use most.

4. General Splitter — Configuration (Processing Tab)

Setting	Value in Class	Meaning
Expression Type	**XPath**, Line Break	XPath for XML input; Line Break for non-XML (CSV / plain text)
XPath Expression	`//employee`	The repeating element to split on (here every `employee`)
Grouping	(blank)	How many records go into each split message
Streaming	☑	Process a large message in chunks
Parallel Processing	☐	Process the split messages at the same time
Stop On Exception	☑	Stop splitting when an error occurs

> **XPath caution (verified in SAP's documentation):** the Splitter's XPath does **not** support these characters: `| + * > < >= <= [ ] @`. So predicates like `employee[2]` (Day 12) won't work here — use a plain path such as `//employee` or `/root/employeeInfo/employee`. For namespaced XML (Day 20), declare the prefix in the iFlow's Runtime Configuration → Namespace Mapping.

Grouping

> If the input payload has **100 records** and the **target system can receive only 10 records per call**, what do we do? **We use Grouping** — if we want to group some set of records together, we use Grouping. *(Grouping = 10 → 10 messages of 10 records each.)*

SAP also lets you point Grouping at a **header or property** instead of a fixed number, so the group size can be set dynamically.

Streaming

Bulk data incoming → tick Streaming → upscales the performance. More precisely (verified): with streaming, CPI **starts processing the first chunks before the whole message is loaded into memory**; leaving it off is **more memory-intensive**. So streaming matters most for big payloads.

Parallel Processing

When you split a message into many messages and want to **execute all of them at the same time**, tick **Parallel Processing**. **By default the Splitter works in a sequential format** (one split after another).

> **Added detail (verified):** once ticked, you also set **Number of Concurrent Processes** (how many threads) and a **Timeout (in seconds)**. **Caution:** if the overall processing exceeds the timeout, the Splitter **stops without raising an exception** and the flow moves on — so unfinished splits can be silently skipped. Also, don't rely on the splits finishing in the original order when running in parallel.

Stop On Exception

> **Scenario:** you are splitting the data and sending it to the target system. The **1st record is processed successfully**; for some other reason the **2nd record fails** (during splitting). What do we have to do?
>
> - **Option A — continue** with the other messages' splitting, or
> - **Option B — end the current message splitting with an error.**
>
> **Stop On Exception ☑ → the splitting will stop** (Option B).

Stop On Exception	What Happens When Record 2 Fails
Selected ☑	Processing stops at the failure; the remaining records are not processed; the message ends with an error
Not selected ☐	The next splits are still executed. Without an Exception Subprocess the overall message still ends **Failed**; if an **Exception Subprocess** handles the error and ends with an **End Message**, processing continues normally (verified in SAP's "Splitter with exception handling" guide, Days 15–16)

Line Break (Non-XML Input)

Basically: you are getting a **CSV file or plain text file** and you want to **split the data in single, single lines** → use **Line Break** — **each line becomes a separate message.**

> **Watch the header row:** a SAP Community example reports that the General Splitter with Line Break treats the **first line as a header** (splitting starts from line 2). If your CSV has a header, check in Trace how your first line is handled.

5. Iterating Splitter

> **Iterating Splitter:** it has the **same functionality as the General Splitter**, but the **output comes without the root tags**.

	General Splitter	Iterating Splitter
Output	**Root tag** / header (enveloping) elements are kept in every split message	**No root tag** — only the required node's output; no parent/root elements

Illustrative Output for Your Employee Payload (Split on `//employee`)

Splitter	Message 1
General	`<root><employeeInfo><employee><EmpID>1</EmpID><status>Active</status></employee></employeeInfo></root>`
Iterating	`<employee><EmpID>1</EmpID><status>Active</status></employee>`

> **Verified, with two extra facts:**
>
> - SAP defines **enveloping elements** as the elements **above and including the split point**. With the General Splitter, elements that come **after** the split point on the same level are **not** copied into the result messages (they go missing) — worth knowing if your payload has a trailer/footer element.
> - The Iterating Splitter has a **third expression type, "Token"**, besides XPath and Line Break.

6. Useful Properties After a Split (Added)

Property	Use
`${property.CamelSplitIndex}`	The position of the current split (0, 1, 2 …)
`${property.CamelSplitSize}`	Total number of splits

Example from SAP: a file name such as `split_${exchangeId}_Index${property.CamelSplitIndex}` gives every split its own unique file name when writing to an SFTP server.

If the splits must be **merged back** into one message after processing, add a **Gather** step (Day 9's routing list).

7. Quick Interview Answers

Question	Short Answer
Why use a Splitter?	To break one composite/huge message into individual messages — e.g., when the target accepts one record per call, or the Router must decide per record
General vs. Iterating Splitter?	General keeps the enveloping/root elements in each split; Iterating outputs just the split node without them
What is Grouping?	Number of records per split message (100 records, group of 10 → 10 messages)
What does Streaming do?	Processes a large message in chunks to save memory and improve performance
Default processing mode?	Sequential; tick Parallel Processing to run splits concurrently
What does Stop On	Stops the splitting at the first failed record; otherwise the next splits still run

Exception do?	
How do you split a CSV?	Expression Type = Line Break — each line becomes a message
How do you merge splits back?	Use a Gather step after the processing